

Podział pracy

Zespół projektowy nr 4 · Warszawa, 11 czerwca 2026

Pracę podzielono *pionowo*: każdy mechanizm semantyczny klasy *DS4* (model dziedziny, akcje proste, akcje złożone, proces i kwerendy, formuły, interfejsy) ma jednego właściciela, prowadzącego go od formalizacji w teorii, przez implementację modułów, po testy; granice części pokrywają się z granicami modułów projektu *Ds4.Core*.

Wkład zestawiono w czterech punktach kontrolnych: **PK1** (teoria wstępna, 26.03), **PK2** (pełna teoria, 23.04), **PK3** (prototyp aplikacji, 12.05) oraz **PK4** (wersja końcowa: teoria, aplikacja, testy, dokumentacja, 11.06). Poza ogólną koordynacją (W. Jasiński) wyróżniono dwie role wiodące: **Krzysztof Gryboś** — lider części teoretycznej (PK2) oraz **Jarosław Bieniek** — lider części programistycznej (PK3). Wkład wszystkich członków był zrównoważony.

Podział prac według punktów kontrolnych

Wojciech Jasiński (koordynator)

Obszar: warstwa API i integracja (*Api/Ds4Facade*, *Api/Contracts*), struktura repozytorium, przykłady demonstracyjne i wydanie aplikacji.

- **PK1:** koordynacja zespołu, ustalenie zakresu klasy *DS4* (warunki Z1–Z7) i struktury dokumentu teorii wstępnej; podział mechanizmów semantycznych między członków.
- **PK2:** złożenie pełnej teorii w $\text{\LaTeX}$, ujednolicenie notacji (Σ , Σ_0 , *Res*) między częściami, przygotowanie wydruku.
- **PK3:** szkielet rozwiązania i układ projektów (*Ds4.Core/Cli/Gui/CrossGui*), fasada *Ds4Facade* spinająca parser i semantykę w jeden punkt wejścia, zestaw przykładów demonstracyjnych.
- **PK4:** reorganizacja teorii do wersji v2 wg uwag recenzji, redakcja dokumentu technicznego, ujednolicenie kontraktu wyniku (*TAK/NIE*, uzasadnienie, $|\Sigma|/|\Sigma_0|$, *trace*, opis grafu DOT), kompilacja i wydanie aplikacji wieloplatformowej, scalanie gałęzi i higiena repozytorium.

Krzysztof Gryboś (lider części teoretycznej)

Obszar: język kwerend i procesów (*Semantics/ProcessEvaluator*, *Semantics/QueryEvaluator*, *Semantics/ExecutionTrace*, *Query/QueryModel*, *Parser/ProcessParser*, *Parser/QueryParser*).

- **PK1:** wstępny opis języka kwerend i rodzajów pytań (*executable*, *accessible*, *cel*), pojęcie procesu jako sekwencji akcji.
- **PK2:** formalna semantyka *necessary/possibly* dla wielu stanów początkowych, definicja procesu i śladu wykonania, kwantyfikacja po Σ_0 .
- **PK3:** silnik procesów i kwerend — budowa drzewa wykonań (poziom *i* to $\text{Res}(\mathbb{A}_i, \cdot)$), oznaczanie gałęzi uciętych ($\text{Res} = \emptyset$) oraz ewaluacja czterech wariantów pytań na tym drzewie.
- **PK4:** korekta semantyki *possibly* do postaci „dla każdego $\sigma \in \Sigma_0$ istnieje ścieżka” (kwantyfikacja uniwersalna po Σ_0), rozdzielenie **after/observable** jako warunków ważności *M2* zamiast filtrów Σ_0 , testy procesów i wszystkich wariantów kwerend.

Jarosław Bieniek (lider części programistycznej)

Obszar: parser dziedziny i obsługa wejścia (`Parser/DomainParser`, `Parser/ParseException`) oraz interfejs wiersza poleceń (`Ds4.Cli/Program`).

- **PK1:** przegląd algorytmiczny — zarys generacji modelu i ewaluacji, identyfikacja głównych kroków przetwarzania.
- **PK2:** pseudokod algorytmów (`Res`, wykrywanie konfliktów, ewaluacja kwerend) w części teoretycznej.
- **PK3:** parser dziedziny (fluenty, akcje proste, zdania **causes/releases/impossible/initially/always**), wczytywanie plików wejściowych oraz interfejs wiersza poleceń (`Ds4.Cli`).
- **PK4:** obsługa błędnego wejścia i czytelne komunikaty (`ParseException`), rozpoznawanie nazw akcji podobnych do słów kluczowych, ujednolicenie składni słownej operatorów, przeniesienie algorytmów do dokumentu technicznego, testy parsera i regresyjne (m.in. warunkowe **causes**).

Filip Filipkowski

Obszar: model dziedziny i semantyka akcji prostych (`Model/Domain`, `Model/State`, `Model/Symbols`, `Semantics/SimpleActionEngine`, `Semantics/ModelBuilder`).

- **PK1:** wstępny opis języka akcji i zdań **causes/releases**, intuicja efektów i inercji.
- **PK2:** składnia i semantyka akcji prostych (efekty, inercja, minimalizacja zmian, zwolnienia fluentów, niewykonalność), definicja modelu dziedziny.
- **PK3:** silnik akcji prostych — operator $\text{Res}(A, \sigma)$ z minimalizacją zbioru zmienianych literalów, filtrowanie wartościowań do Σ (**always**) i zbioru początkowego Σ_0 (**initially**, `ModelBuilder`), reguły **impossible**.
- **PK4:** dopracowanie inercji i ramifikacji wg uwag recenzji, rozdzielenie roli **initially** (źródło Σ_0) od **always** (ograniczenia integralności), testy modelu oraz Σ/Σ_0 i akcji prostych.

Sebastian Rydz

Obszar: akcje złożone — konflikty i dekompozycja (`Semantics/CompositeActionEngine`, `Semantics/ConflictDetector`, `Semantics/DecompositionGenerator`) oraz reprezentacja zmian i wyniku przejścia używana przez detekcję konfliktów (`Semantics/LiteralChange`, `Semantics/TransitionModel`).

- **PK1:** wstępny opis akcji złożonych i problemu współbieżnego wykonywania akcji prostych.
- **PK2:** formalizacja konfliktów między parami akcji (przypadki sprzecznych efektów **causes/releases**) oraz dekompozycji jako maksymalnych bezkonfliktowych podzbiorów.
- **PK3:** implementacja detekcji konfliktów w stanie σ i generacji maksymalnych dekompozycji; operator $\text{Res}(A, \sigma)$ akcji złożonej jako suma wyników po dekompozycjach (źródło niedeterminizmu $Z3$).
- **PK4:** doprecyzowanie definicji konfliktu wg teorii v2 (dwa przypadki, usunięcie reguły nakładania się), testy konfliktów, dekompozycji i akcji złożonych.

Jakub Pietrzak

Obszar: interfejsy graficzne i wizualizacja wykonań (`Ds4.Gui`, `Ds4.CrossGui`, `Semantics/TraceGraphRenderer`), generacja przestrzeni stanów (`Semantics/StateGenerator`) oraz redakcja teorii.

- **PK1:** współudział w redakcji teorii wstępnej i przeglądzie literatury (języki akcji).
- **PK2:** podsumowanie i redakcja pełnej teorii, spójność terminologii i odsyłaczy.
- **PK3:** interfejs graficzny w dwóch wariantach (`Ds4.Gui` — WinForms, `Ds4.CrossGui` — Avalonia, Linux/macOS), generacja przestrzeni stanów (wyliczenie wszystkich wartościowań fluentów, `StateGenerator`) oraz renderowanie grafu wykonań do formatu Graphviz DOT (`TraceGraphRenderer`).
- **PK4:** testy pełnej semantyki (scenariusze end-to-end) i źródeł GUI (`CrossGuiSourceTests`), dopracowanie prezentacji wyników (TAK/NIE, uzasadnienie, podgląd grafu).

Sandra Adamiec

Obszar: formuły zdaniowe i ich postać normalna (`Formula/IFormula`, `Formula/FormulaNodes`, `Formula/FormulaTools`, `Parser/FormulaParser`) oraz instrukcja użytkownika.

- **PK1:** wprowadzenie i wstępny opis formuł zdaniowych (fluenty jako zmienne, spójniki logiczne).
- **PK2:** definicja formuł, literalów i postaci normalnej DNF, przegląd notacji używanej w zdaniach języka.
- **PK3:** reprezentacja formuł jako drzewa (`FormulaNodes`), parser formuł oraz konwersja do DNF (`FormulaTools`) wykorzystywana przy ewaluacji warunków.
- **PK4:** instrukcja użytkownika, ujednolicenie składni słownej operatorów logicznych, testy formuł, postaci DNF i pokrycia konstrukcji języka.

Autorstwo testów

Zestaw testów (projekt `Ds4.Tests`, framework xUnit) liczy **114 testów** w 18 plikach. Każdy plik testów dotyczy przede wszystkim jednego obszaru, więc testy przypisano do właścicieli obszarów — tak samo jak implementację. Kolumna *rodzaj*: **J** — jednostkowy, **I** — integracyjny lub systemowy, **R** — regresyjny; kolumna *n* — liczba testów w pliku.

Plik testów	Zakres	rodz.	n
<i>Filip Filipkowski — model dziedziny i akcje proste (18)</i>			
ModelTests	stan, fluenty inercyjne, inferencja fluentów	J	4
StateAndModelBuilderTests	generacja wartościowań, filtry Σ/Σ_0 , walidacja dziedziny	J	6
SimpleActionEngineTests	Res: efekty, inercja, zwolnienia, impossible , ramifikacje	J	8
<i>Sebastian Rydz — akcje złożone (8)</i>			
CompositeActionTests	konflikty, dekompozycje, niedeterminizm akcji złożonej	J	8
<i>Krzysztof Gryboś — proces i kwerendy (13)</i>			
ProcessAndQueryTests	drzewo wykonania, cztery warianty kwerend, ślad	J	9
PossiblySemanticsTests	<i>possibly/necessary</i> z kwantyfikacją po Σ_0	J	4
<i>Sandra Adamiec — formuły i pokrycie języka (21)</i>			
FormulaTests	ewaluacja, precedencja, DNF, prawa de Morgana	J	7
WordOnlySyntaxTests	wyłączenie słowne operatory, odrzucanie symboli	J	4
LanguageCoverageTests	pokrycie wszystkich konstrukcji języka i parserów	J	10
<i>Jarosław Bieniek — parser i obsługa wejścia (19)</i>			
ParserTests	parsowanie dziedziny/kwerend/procesów, odrzucanie błędów	J	9
KeywordLikeActionNameRegressionTests	nazwy akcji podobne do słów kluczowych	R	4
ConditionalCauseRegressionTests	warunkowe causes (parser $\rightarrow$ kwerenda)	R	6
<i>Jakub Pietrzak — interfejs graficzny i pełna semantyka (12)</i>			
CrossGuiSourceTests	źródła GUI Avalonia, panel przykładów, graf DOT	I	6
FullSemanticsIntegrationTests	zgodność wszystkich mechanizmów end-to-end	I	6
<i>Wojciech Jasiński — integracja, fasada, przykłady (23)</i>			
FacadeAndExamplesTests	ładowanie przykładów, zgodność wyników TAK/NIE	I	7
ScenarioExamplesTests	scenariusze fabularne, filozofowie	I	3
CacheAndGraphTests	pamięć podręczna fasady, czytelny graf DOT	I	6
StaticProjectTests	higiena repozytorium, struktura rozwiązania	I	7
Razem			114